ENTERPRISE TECHNICAL SPECIFICATION & PROJECT REPORT

AUTOMATION-CGI WAREHOUSE MANAGEMENT SYSTEM

A High-Performance, Multi-Tenant Full-Stack WMS Platform Integrating Real-Time Physical Location Tracking, Bill of Materials (BOM), Conversational AI Intelligence, Optical Scanning, and Desktop Deployment.



Production Engineering Project & Technical Architecture Report

System Release v1.0.0 • Production Build

- ✓ React 19 & Vite SPA
- ✓ Node.js & Express 5 Layered Backend
- ✓ PostgreSQL Multi-Tenant DB
- ✓ Conversational Warehouse AI
- ✓ ECharts Dynamic Visualizations
- ✓ Razorpay Payment Gateway
- ✓ QR/Barcode Optical Scanner
- ✓ Electron Desktop Client

PREPARED BY / LEAD ENGINEER

Rishabh

PROJECT REPOSITORY

Automation-CGI (Enterprise WMS)

DATABASE ARCHITECTURE

PostgreSQL 18.4 / Supabase Multi-Tenant

DOCUMENT DATE & STATUS

March 2026 • Final Implementation Report

TABLE OF CONTENTS

INDEX

1. Executive Summary & Abstract — Page 3
2. Problem Statement & Motivation — Page 3
3. Project Scope & Objectives — Page 4
4. Technology Stack Breakdown — Page 4
5. System Architecture & Design — Page 5
6. Core Functional Modules (6.1–6.7) — Page 6
7. Advanced Intelligence & Integration (6.8–6.13) — Page 7
8. Database Schema & Data Dictionary — Page 8
9. RESTful API Endpoint Specifications — Page 9
10. Security, ACID Transactions & Performance — Page 10
11. Deployment, Desktop Client & Verification — Page 10
12. Future Roadmap & Conclusion — Page 11

1. EXECUTIVE SUMMARY & ABSTRACT

SECTION 1

Automation-CGI is an enterprise-grade, full-stack Warehouse Management System (WMS) engineered to solve the complex operational challenges of modern supply chain and fulfillment hubs. Combining a high-speed web and native desktop interface with an event-driven, layered REST backend and PostgreSQL database, the system governs the entire physical and transactional lifecycle of inventory items—from supplier procurement and multi-tiered Bill of Materials (BOM) management to physical warehouse slotting (Aisles, Racks, Bins), customer sales orders, and automated payment settlement.

Beyond standard warehousing operations, Automation-CGI incorporates an innovative **Conversational Warehouse AI Engine** with multi-turn context memory and hybrid rule/LLM tool dispatching, enabling warehouse personnel to query inventory health, stockouts, and order metrics in natural language. Equipped with interactive ECharts analytics (Heatmaps, Sankey flow diagrams, Gauges), live webcam/mobile QR code scanning, multi-tenant organization isolation, and PDF/Excel export engines, Automation-CGI represents a state-of-the-art solution designed for high throughput, sub-second latency, and rock-solid operational reliability.

2. PROBLEM STATEMENT & INDUSTRIAL MOTIVATION

SECTION 2

Modern logistics centers manage tens of thousands of SKUs across fluctuating customer demands. Conventional warehouse operations routinely encounter critical bottlenecks:

- **Physical Discrepancies & Blind Spots:** Disconnected spreadsheets and manual logs cause variances between reported software inventory and actual physical stock on warehouse shelves.

- **Lack of Granular Spatial Tracking:** Standard inventory software tracks quantities by facility but fails to specify precise physical coordinates (Aisle, Rack, Bin), resulting in worker search latency and picking inefficiencies.
- **Stock Reservation & Allocation Confusion:** Failing to differentiate between Total On-Hand, Reserved Stock for pending orders, and Damaged Stock leads to accidental overselling, delayed shipments, or unexpected stockouts.
- **Rigid Analytics & Delayed Action:** Warehouse supervisors spend hours exporting static CSVs to determine reorder needs, lacking dynamic visibility into supplier lead times and warehouse stock health.
- **Siloed Financial & Procurement Workflows:** Disconnected procurement systems delay restocking when purchase orders are manually communicated without integrated payment and supplier verification.

> **Key Solution Impact:**
> Automation-CGI unifies physical location slotting, multi-level BOMs, ACID-compliant sales transactions, integrated Razorpay checkouts, and natural language AI query tools into one unified platform, eliminating data silos and operational latency.

3. PROJECT SCOPE & SYSTEM OBJECTIVES

SECTION 3

The project encompasses both functional capabilities for warehouse operators and technical requirements for system scalability, security, and portability:

SPATIAL PRECISION
Aisle-Rack-Bin
Physical sub-location mapping for fast item retrieval

DATA CONSISTENCY
ACID Compliant
PostgreSQL transactions with rollback safety

INTELLIGENCE
Hybrid AI
Rule-based intent routing + LLM fallback

CROSS-PLATFORM
Web & Desktop
Vite SPA + Native Electron packaging

3.1 Core Objectives

- **Real-time Stock Visibility:** Track `available_quantity = total - reserved - damaged` with instantaneous updates upon order placement and fulfillment.
- **Automated Reorder Safeguards:** Alert managers and generate Purchase Orders whenever stock drops below designated reorder thresholds.
- **Conversational Natural Language Interface:** Allow non-technical floor managers to query inventory, look up suppliers, and generate charts via an AI assistant.
- **Multi-Tenancy & RBAC:** Securely partition enterprise data across organizations with granular Role-Based Access Control (Admins, Managers, Staff).
- **Dual Deployment:** Support both browser-based operations and standalone offline-tolerant desktop execution via Electron.

4. TECHNOLOGY STACK SPECIFICATIONS

SECTION 4

Automation-CGI leverages a modern, decoupled tech stack designed for high throughput, maintainability, and responsiveness:

TIER / DOMAIN	TECHNOLOGIES USED	KEY ROLE & ARCHITECTURE RESPONSIBILITY
Frontend Core	React 19, Vite, React Router v7	Single-page application (SPA), route guards, component hierarchy, instant HMR.
Styling & Motion	Tailwind CSS v4, Framer Motion	Responsive modern interface, animated transitions, modal overlays, theme tokens.
Data Visualization	Apache ECharts, Recharts	Dynamic heatmaps, inventory stock distribution, monthly sales revenue charts.
Hardware / Optical	html5-qrcode, qrcode.react	Live webcam/mobile barcode and QR scanning, client-side QR label generation.
Backend API	Node.js v24, Express v5	RESTful API layer, layered Controller-Service-Repository design pattern.

TIER / DOMAIN	TECHNOLOGIES USED	KEY ROLE & ARCHITECTURE RESPONSIBILITY
Database & ORM	PostgreSQL 18.4, Supabase, `pg.Pool`	Relational schema, foreign keys, multi-tenant isolation, connection pooling.
Auth & Security	JWT, Bcrypt, Nodemailer	Salted password hashing, stateless tokens, email OTP verification flows.
Financial Services	Razorpay SDK	Order creation, client checkout integration, cryptographic signature verification.
AI & Analytics	Custom Rule Engine, Ollama / OpenAI	Intent extraction, multi-turn session store, domain tool dispatching.
Desktop Packaging	Electron v43, electron-builder	Native Windows executable wrapping frontend and spawning backend Node process.

5. SYSTEM ARCHITECTURE & HIGH-LEVEL DESIGN

SECTION 5

The system is structured according to an Enterprise Multi-Tier Layered Architecture that guarantees separation of concerns, modular testability, and high horizontal scalability:



5.1 Layered Backend Architecture Pattern

The backend strictly enforces the **Controller-Service-Repository** design pattern:

- **Controllers (e.g., `orderController.js`):** Accept incoming HTTP requests, validate query parameters and JSON payloads, and return standardized JSON responses (`{ success, data, message }`).
- **Services (e.g., `paymentService.js`, `aiService.js`):** Implement core business policies, mathematical validations, OTP email transmissions, and multi-step workflows.
- **Repositories (e.g., `inventoryRepository.js`):** Direct data access layer executing optimized SQL queries with parameterized variables (`$1, $2, ...`) preventing SQL injection attacks.

6. DETAILED SYSTEM MODULES & FUNCTIONAL IMPLEMENTATION

SECTION 6

6.1 Authentication & Multi-Tenant Access Control (RBAC)

User authentication is secured using JSON Web Tokens (JWT) and Bcrypt cryptographic hashing (salt rounds: 10). The registration and password reset workflows feature an integrated two-factor verification mechanism powered by `otp-generator` and `nodemailer`. Users receive a time-expiring 6-digit numeric OTP via SMTP. The client verifies the token before activating the user account or permitting password modification. Protected routes in React (`ProtectedRoute.jsx`) ensure unauthorized access is blocked.

6.2 Executive Dashboard & Live Business Intelligence

The executive dashboard provides high-level situational awareness through parallel database aggregations (`Promise.all`):

- **KPI Summary Cards:** Real-time counts of Total Products, Low-Stock Warnings (items below reorder thresholds), Total Orders, Delivered Revenue, and Active Suppliers.
- **Warehouse Stock Health Heatmap:** Visual matrix depicting inventory density and stock levels across multiple warehouse facilities.
- **Recent Activity & Status Badges:** Chronological feed of recent customer orders and purchase orders with dynamic status tracking (`PENDING` , `PROCESSING` , `SHIPPED` , `DELIVERED`).

6.3 Granular Physical Inventory & Location Slotting

Unlike simplistic inventory trackers, Automation-CGI incorporates a 3-dimensional physical coordinate system: **Aisle, Rack, and Bin** linked through the `locations` table. The system calculates:

```
available_quantity = quantity - COALESCE(reserved_quantity, 0) - COALESCE(damaged_quantity, 0)
```

Warehouse operators can rapidly adjust damaged quantities, update stock counts via modals, and inspect unit costs and selling prices for precise warehouse valuation.

6.4 Product Master Catalog & Bill of Materials (BOM)

The product module maintains SKU definitions, descriptions, barcodes, category classifications, and reorder levels. Additionally, the system supports assembly manufacturing through the **Bill of Materials (BOM)** engine. The `bom` and `bom_items` tables define the multi-component ingredients required to manufacture a composite finished product, complete with required quantities and unit measurements.

6.5 Sales Order Lifecycle & ACID Transactional Fulfillment

Orders encapsulate multi-line items associated with customers. Order creation executes within a strict PostgreSQL database transaction:

```
BEGIN;
-- 1. Insert parent order record
INSERT INTO orders (customer_id, status, total_amount) VALUES ($1, 'PENDING', $2) RETURNING *;
-- 2. Insert line items in loop
INSERT INTO orderitems (order_id, product_id, quantity, unit_price, total_price) VALUES ...;
COMMIT; -- Or ROLLBACK on error
```

Upon order shipment, reserved quantities are cleared, physical stock is decremented, and revenue is recorded upon `DELIVERED` status.

6.6 Procurement & Purchase Order (PO) Management

When inventory levels drop below thresholds, procurement managers generate Purchase Orders mapped to approved suppliers. The PO lifecycle transitions through `PENDING`, `APPROVED`, `RECEIVED`, or `CANCELLED`, allowing seamless restocking into designated warehouse locations.

6.7 Supplier & Vendor Directory

Maintains complete vendor profiles including supplier names, contact emails, phone numbers, addresses, and transaction histories, ensuring reliable supply chain continuity.

7. ADVANCED INTELLIGENCE & HARDWARE INTEGRATIONS

SECTION 7

6.8 Conversational Warehouse AI Assistant

Automation-CGI features a native, proprietary **Warehouse AI Engine** (`/api/ai/chat`) capable of answering real-time business queries without requiring SQL knowledge. The architecture implements a hybrid multi-tier intelligence pipeline:

Hybrid AI Execution Pipeline:

1. **Session Context Resolver:**
   Recalls multi-turn context (e.g., user asks "What are my low stock items?" followed by "Show me the suppliers for those").
2. **Rule-Based Intent Analyzer:**
   High-speed keyword and pattern scoring matching intents (`INVENTORY_QUERY` , `LOW_STOCK` , `ORDER_STATUS` , `PRICE_CHECK` , `SUPPLIER_LOOKUP`).
3. **LLM Fallback Router:**
   Natural language prompts that do not match exact rule patterns are routed to a local or cloud LLM (Ollama / OpenAI) for zero-shot entity extraction.
4. **Domain Tool Dispatcher:**
   Maps resolved intents to backend repositories (`inventoryTool` , `orderTool` , `productTool`) to retrieve actual live database records.
5. **Natural Language Generator:**
   Formats the raw database rows into an intuitive, readable response returned to the frontend chat UI.

6.9 AI-Driven Dynamic Chart Generation

Inside the dashboard, users can request visualizations in natural language (e.g., "*Show me a heatmap of warehouse stock*" or "*Generate an inventory flow sankey diagram*"). The `AIChart` component parses the request and dynamically renders:

- **Heatmaps:** Stock density across facilities.
- **Sankey Diagrams:** Visual inventory flow and stock movements.
- **Scatter Plots & Radar Charts:** Category distributions and stock balance metrics.
- **Gauge Health Meters:** Real-time warehouse utilization and stock health indexes.

6.10 Optical Barcode & QR Code Scanning

Floor workers can perform instant stock lookups and physical inventory counts using the integrated camera scanner powered by `html5-qrcode` . Concurrently, the product catalog dynamically generates scannable QR codes using `qrcode.react` for printing onto physical warehouse bin labels.

6.11 Integrated Razorpay Payment Gateway

For B2B orders and customer checkouts, Automation-CGI integrates the Razorpay SDK. The server generates an authenticated `order_id`, prompts the interactive Razorpay checkout modal in React, and verifies payment authenticity on the server using cryptographic HMAC SHA-256 signature matching before marking orders as `PAID`.

6.12 Operational Reporting & Data Export Engine

The reports feature provides customizable filtering by date range, warehouse facility, and status. Reports can be immediately exported in two enterprise formats:

- **PDF Export:** Rendered directly via `jspdf` and `jspdf-autotable` with formatted headers and page counters.
- **Excel Spreadsheet:** Exported using `xlsx` and `file-saver` for corporate spreadsheet workflows.

6.13 Cross-Platform Native Desktop Client (Electron)

To accommodate rugged warehouse terminals that operate offline or in standalone kiosk mode, Automation-CGI is bundled into a native Windows executable via Electron. The `electron.js` main process automatically spawns the Node.js backend server in a background child process and mounts the production Vite build in a desktop window (1400x900).

8. DATABASE SCHEMA & DATA DICTIONARY

SECTION 8

The relational database is architected on PostgreSQL 18.4 with 16 interconnected tables maintaining strict foreign key constraints, sequence generators, and multi-tenant organization identifiers:

TABLE NAME	PRIMARY KEY	FOREIGN KEY RELATIONSHIPS	PURPOSE & CORE ATTRIBUTES
`organizations`	`id (PK)`	-	Multi-tenant root entity isolating company accounts, names, and creation timestamps.
`warehouses`	`id (PK)`	`organization_id -> organizations(id)`	Physical storage facilities, addresses, capacity, and active operating status.
`locations`	`id (PK)`	`warehouse_id -> warehouses(id)`	Spatial coordinate mapping inside warehouse: `aisle`, `rack`, and `bin`.
`products`	`id (PK)`	`organization_id -> organizations(id)`	Master item catalog: `sku`, `name`, `category`, `reorder_threshold`.
`inventory`	`id (PK)`	`product_id -> products(id)` `location_id -> locations(id)`	Stock counts: `quantity`, `reserved_quantity`, `damaged_quantity`, `unit_cost`.
`bom`	`id (PK)`	`product_id -> products(id)`	Bill of Materials header specifying assembly version and active status.
`bom_items`	`id (PK)`	`bom_id -> bom(id)` `component_product_id -> products(id)`	Bill of Materials line items: required component products, quantities, and units.
`suppliers`	`id (PK)`	`organization_id -> organizations(id)`	Vendor profiles: `name`, `contact_name`, `email`, `phone`, `address`.
`purchaseorders`	`id (PK)`	`supplier_id -> suppliers(id)`	Procurement orders: `order_number`, `status`, `total_amount`, `order_date`.
`purchaseorderitems`	`id (PK)`	`purchase_order_id -> purchaseorders(id)` `product_id -> products(id)`	Inbound PO items: `quantity`, `unit_price`, `total_price`.
`orders`	`id (PK)`	`customer_id -> users(id)`	Customer sales orders: `status` (PENDING, DELIVERED), `total_amount`.
`orderitems`	`id (PK)`	`order_id -> orders(id)` `product_id -> products(id)`	Sales order line items: `quantity`, `unit_price`, `total_price`.
`users`	`id (PK)`	`organization_id -> organizations(id)`	System accounts: `name`, `role` (Admin, Staff), `email`, `password` (Bcrypt).
`otp_verification`	`id (PK)`	-	Two-factor tokens: `email`, `otp`, `expires_at`, `is_verified`.

TABLE NAME	PRIMARY KEY	FOREIGN KEY RELATIONSHIPS	PURPOSE & CORE ATTRIBUTES
`payments`	`id (PK)`	`order_id -> orders(id)`	Financial logs: `provider` (RAZORPAY), `provider_order_id`, `amount`, `status`.
`stockmovements`	`id (PK)`	`product_id -> products(id)` `location_id -> locations(id)`	Immutable audit log: stock transfer tracking, timestamps, and user associations.

9. RESTFUL API SPECIFICATIONS

SECTION 9

The backend exposes a comprehensive RESTful API adhering to HTTP semantics and standardized JSON response payloads:

MODULE	METHOD	ENDPOINT PATH	DESCRIPTION & PARAMETERS
Auth	`POST`	`/api/auth/register`	Creates user account with hashed password and generates initial verification OTP.
Auth	`POST`	`/api/auth/login`	Validates credentials via Bcrypt; returns signed JWT bearer token and user metadata.
Auth	`POST`	`/api/auth/verify-otp`	Validates 6-digit email OTP against `otp_verification` table.
Dashboard	`GET`	`/api/dashboard`	Returns aggregated KPI metrics, categories breakdown, heatmap matrix, and recent orders.
Inventory	`GET`	`/api/inventory`	Fetches all inventory joined with physical coordinates (aisle, rack, bin) and status.
Inventory	`PUT`	`/api/inventory/:id`	Updates inventory quantities, reserved levels, or damaged units.
Products	`GET` / `POST`	`/api/products`	Lists active SKUs or registers new catalog products with reorder thresholds.
Products	`PUT` / `DELETE`	`/api/products/:id`	Modifies product attributes or removes obsolete catalog items.
Orders	`GET` / `POST`	`/api/orders`	Retrieves order history or executes multi-line order creation within ACID transaction.
Purchase Orders	`GET` / `POST`	`/api/purchaseorders`	Fetches procurement records or issues new restock PO to a designated supplier.
Purchase Orders	`PUT`	`/api/purchaseorders/:id/status`	Updates PO status to `RECEIVED` and triggers automatic stock replenishment.
Suppliers	`GET` / `POST`	`/api/suppliers`	Manages external vendor directory and contact records.
Payments	`POST`	`/api/payments/create`	Initializes a Razorpay payment order for an existing sales order.
Payments	`POST`	`/api/payments/verify`	Verifies HMAC SHA-256 signature; updates payment status to `PAID`.
AI Assistant	`POST`	`/api/ai/chat`	Accepts `{ prompt, sessionId }`, executes intent resolution, and returns conversational response.

MODULE	METHOD	ENDPOINT PATH	DESCRIPTION & PARAMETERS
Reports	`POST`	`/api/report`	Generates filterable report dataset across inventory, sales, or supplier metrics.

10. SECURITY, RELIABILITY & CONCURRENCY

SECTION 10

10.1 Cryptographic & Application Security

- **Password Security:** User credentials are protected using salted Bcrypt hashes before persistence, ensuring passwords are never stored in plaintext.
- **Stateless Authentication:** Sensitive routes require an `Authorization: Bearer <JWT>` header verified via `authMiddleware.js`.
- **SQL Injection Elimination:** All database queries in the repository tier strictly utilize parameterized placeholders (`$1, $2, ...`), precluding injection vectors.
- **Financial Signature Verification:** Razorpay payments are verified on the backend by calculating `HMAC-SHA256(order_id + "|" + payment_id, secret)`, preventing payment spoofing.

10.2 Concurrency & Transaction Management

Inventory allocation and multi-item orders are susceptible to race conditions. Automation-CGI guarantees atomicity by wrapping multi-row operations inside explicit database transactions (`BEGIN` ... `COMMIT` / `ROLLBACK`). Connection pooling through `pg.Pool` maintains up to 5 concurrent active connections with automated keepalive and 30-second idle timeouts.

11. HARDWARE SPECIFICATIONS & DEPLOYMENT

SECTION 11

Automation-CGI supports versatile deployment topologies adapted to cloud hosting and warehouse floor kiosks:

DEPLOYMENT MODE	ENVIRONMENT / STACK	LAUNCH COMMAND	OPERATIONAL USE CASE
Full-Stack Development	Concurrent Node + Vite dev servers	`npm run dev`	Local feature development, hot-module reloading, debugging.
Native Desktop Client	Electron + Production Web Build	`npm run desktop`	Rugged warehouse floor terminals, barcode scanning kiosks, standalone workstations.
One-Click Batch Script	Windows Shell Batch File	`start-warehouse.bat`	Instant operator launch without terminal command knowledge.
Production Executable	Electron-builder NSIS Installer	`npm run dist`	Generates `.exe` installer for rapid enterprise IT provisioning.

Environment Variables Configuration (`.env`):

Configures `PORT=8000`, `DATABASE_URL` (PostgreSQL SSL connection string), `JWT_SECRET`, `EMAIL_USER` & `EMAIL_PASS` (Gmail SMTP credentials), and `RAZORPAY_KEY_ID` & `RAZORPAY_KEY_SECRET`.

12. FUTURE ROADMAP & STRATEGIC ENHANCEMENTS

SECTION 12

Building upon the solid architecture of Automation-CGI, the following strategic enhancements are slated for future releases:

- **RFID Portal Gate Integration:** Integrating UHF RFID antenna readers at dock doors for zero-touch pallet scanning during inbound unloading and outbound shipping.
- **Predictive AI Demand Forecasting:** Incorporating historical seasonality algorithms to predict stock replenishment quantities weeks in advance (building on the stubbed `forecastService.js`).
- **IoT Environmental Telemetry:** Connecting temperature and humidity sensors in perishable inventory zones with automated alerts for anomalous conditions.
- **Automated Mobile Robot (AMR) Dispatch:** Transmitting pick-path coordinates (Aisle, Rack, Bin) to automated warehouse mobile picking robots.

13. CONCLUSION & TECHNICAL SUMMARY

SECTION 13

Automation-CGI successfully bridges the gap between traditional warehouse inventory tracking and modern intelligent enterprise platforms. By integrating real-time spatial slotting, multi-level Bill of Materials, ACID-compliant sales fulfillment, integrated Razorpay payments, optical camera-based QR scanning, and a conversational AI intelligence assistant, the system provides an all-in-one solution for high-throughput logistics.

The implementation achieves high technical maturity through its clean separation of concerns (React 19 SPA, Express 5 Layered Backend, PostgreSQL Database, and Electron Desktop Shell). The system stands fully functional, thoroughly documented, and ready for enterprise operational deployment.

Project Author: Rishabh
Application: Automation-CGI Enterprise WMS
Status: Verified & Complete

Generated Format: PDF Document Report
Framework: React 19 • Express 5 • PostgreSQL • Electron
Date: March 2026